VOYANA

AI-Powered Collaborative Travel Ecosystem

DESIGN DOCUMENT

Table of Contents

1. Architecture 4
1.1 Design Approach and Principles 4
1.2 Technology Stack 4
1.3 Key Design Decisions 5
1.4 Logical Architecture Diagram 6
1.5 Deployment Diagram 6
2. Database Schema 6
2.1 Modelling Conventions 7
2.2 ER Diagram A - Identity, Trip and Itinerary 8
2.3 ER Diagram B - Booking, Expense and Payment 9
2.4 ER Diagram C - Collaboration, Content and Notifications 10
2.5 Table Inventory 10
2.6 Indexes, Constraints and Integrity Rules 11
3. Component and Class Diagrams 12
3.1 Class Diagram - Trip and Itinerary Domain 13
3.2 Class Diagram- Booking, Expense and Payment Domain 13
3.3 Sequence Diagram - Generate and Accept an AI Itinerary 14
3.4 Sequence Diagram - Booking with Payment and Expense Write-through 15
3.5 Sequence Diagram - Log an Expense, Split and Settle 16
3.6 State Machine Diagrams 17
3.7 Data Flow Diagram (Level 1) 18
4. REST API Contracts 18
4.1 Contract Conventions 18
4.2 Operation Inventory 19
4.3 Representative Contracts 19
POST /trips - create a trip 19
PATCH /activities/{activityId} - optimistic locking 20
POST /bookings/{bookingId}/confirm - atomic confirmation 20
GET /trips/{tripId}/settlements - simplified transfer set 20
4.4 Error Model and Status Code Usage 20
4.5 Security Design Applied to the Contract 21
5. Wireframes and UI/UX Mockups (final) 22
5.1. Landing Page 22
5.2 Registration 23
5.3 Trip Dashboard 24
5.4 Itinerary Builder with AI Assistant 25
5.5 Group Workspace 26
5.6 Expenses and Settlements 27
5.7 Unified Booking 28

5.8 Administration and Analytics Console 29
5.9 Responsive Behaviour 30

1. Architecture

1.1 Design Approach and Principles

Voyana is built as a Service-Oriented Architecture: twelve independently deployable services behind a single API gateway, each owning its own datastore schema and publishing an OpenAPI contract. The design is driven by five principles, each of which constrains later sections of this document.

Principle	What it means in the design	Where it is enforced
Schema ownership	No service reads or writes another service's tables. Cross-context references store a bare identifier with no database-level foreign key.	Section 2 (dashed ER connectors), Section 3 component diagram
Contract before code	Every interface is agreed as an OpenAPI operation before implementation starts, so services can be built in parallel against a stable shape.	Section 4 and the delivered YAML
No silent mutation	AI output, poll outcomes and partner data are proposals until a human accepts them. Every write records an actor.	Section 3 sequence diagrams, itinerary_changes table
Atomic money	A confirmed booking, its itinerary activity and its expense entry are written in one transaction. A failure leaves no partial record.	Section 3 booking sequence, Section 5 booking state machine
Swappable externals	Partner inventory, payment gateway and LLM provider sit behind interfaces so a sandbox implementation can be replaced without touching a service.	Section 3 class diagram, adapter realisations

1.2 Technology Stack

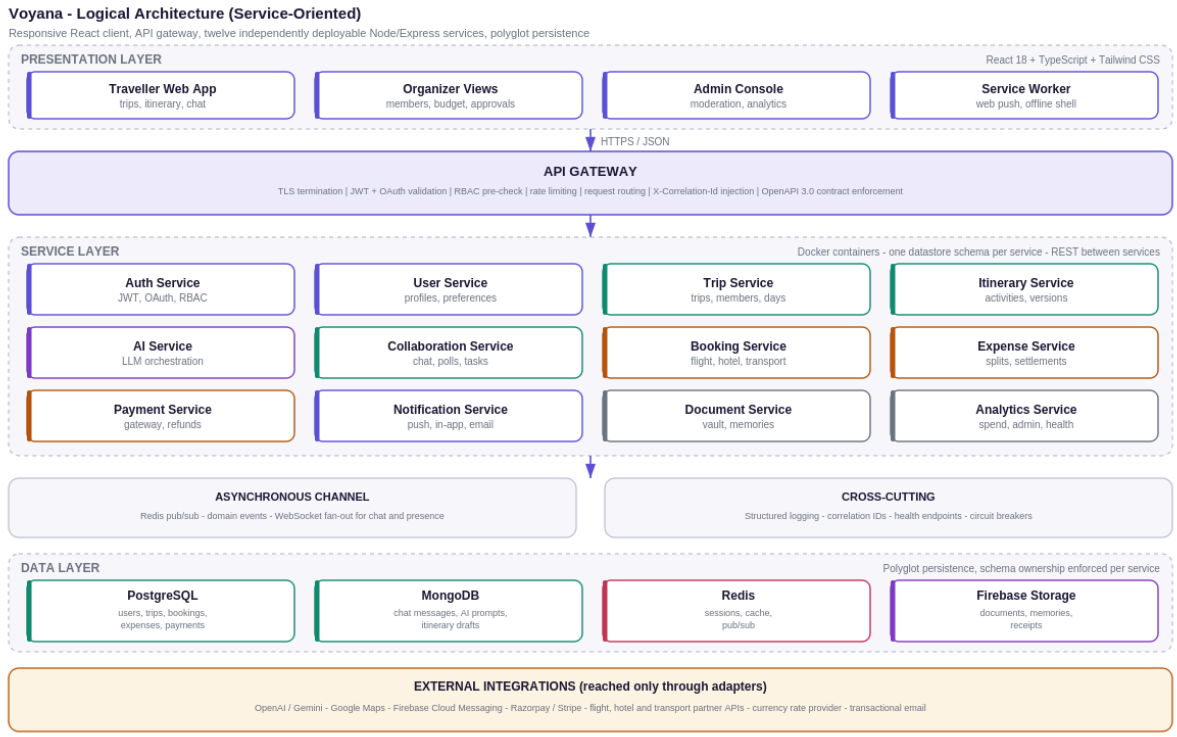
Layer	Technology	Design rationale
Client	React 18, TypeScript, Tailwind CSS, Vite	Single responsive application serves traveller, organizer and admin views; role gating removes the need for a separate build.
Transport	REST over HTTPS, JSON; WebSocket for chat and presence	REST for all state changes so every call is cacheable, retryable and describable in OpenAPI; a socket only where sub-two-second delivery is required.
Services	Node.js 20, Express, TypeScript	One runtime across twelve services keeps the shared middleware (auth, correlation IDs, error envelope) in a single internal package.
Relational store	PostgreSQL 15 (Cloud SQL)	Money, bookings and settlements need transactions, constraints and exact numeric types.
Document store	MongoDB Atlas	Chat messages, AI prompts and itinerary drafts are append-heavy and schema-loose.
Cache and bus	Redis 7 (Memorystore)	Session lookup, read-through caching and pub/sub fan-out for socket delivery. Never the system of record.
Object storage	Firebase Storage and Cloud Storage	Documents and memories served only through signed URLs valid for at most 15 minutes.
AI	OpenAI or Gemini via a provider interface, prompt library versioned in the repository	Provider is replaceable; every request is logged with prompt version, token count and latency.
Runtime	Docker, Cloud Run	Per-service autoscaling; the AI service scales independently of lighter services.

Layer	Technology	Design rationale
Pipeline	GitHub Actions, Artifact Registry	Twelve independent build and deploy pipelines with contract tests in every run.

1.3 Key Design Decisions

#	Decision	Alternative rejected	Reason
D1	Twelve services behind one gateway rather than a modular monolith.	Single Express application with modules.	The course deliverable requires a demonstrable SOA, and eight members can own service boundaries in parallel. The cost is operational overhead, accepted deliberately.
D2	Optimistic locking with an If-Match version token for itinerary edits.	Operational transformation or CRDT for true real-time editing.	Conflict-free concurrent editing is a multi-week problem that no epic is sized for. Optimistic locking plus a visible change log gives correct behaviour at a fraction of the cost.
D3	Booking, activity and expense written in a single database transaction.	Eventual consistency with compensating events.	A booking that exists without a budget entry is the failure users notice most. The three writes live in one service boundary, so a local transaction is available.
D4	Partner inventory behind a PartnerAdapter interface with a mock implementation.	Direct integration with a commercial API.	Commercial inventory requires agreements unavailable to an academic team. The contract is identical, so the sandbox is swappable.
D5	Card capture in the gateway-hosted component only.	Server-side card handling.	Raw card data must never traverse Voyana infrastructure. This removes the entire PCI surface from the design.
D6	Both original and converted expense values stored with the applied FX rate.	Store the converted value only.	A later rate change must never alter a historical figure, otherwise settlements stop reconciling.
D7	Idempotency-Key required on booking, payment and expense writes.	Rely on client retry discipline.	Gateways resend webhooks and users double-click. Idempotency is the only reliable defence against duplicate money records.
D8	AI output is always a draft requiring explicit acceptance.	Apply generated itineraries directly.	A shared plan mutated without a human decision destroys group trust and makes the change log meaningless.

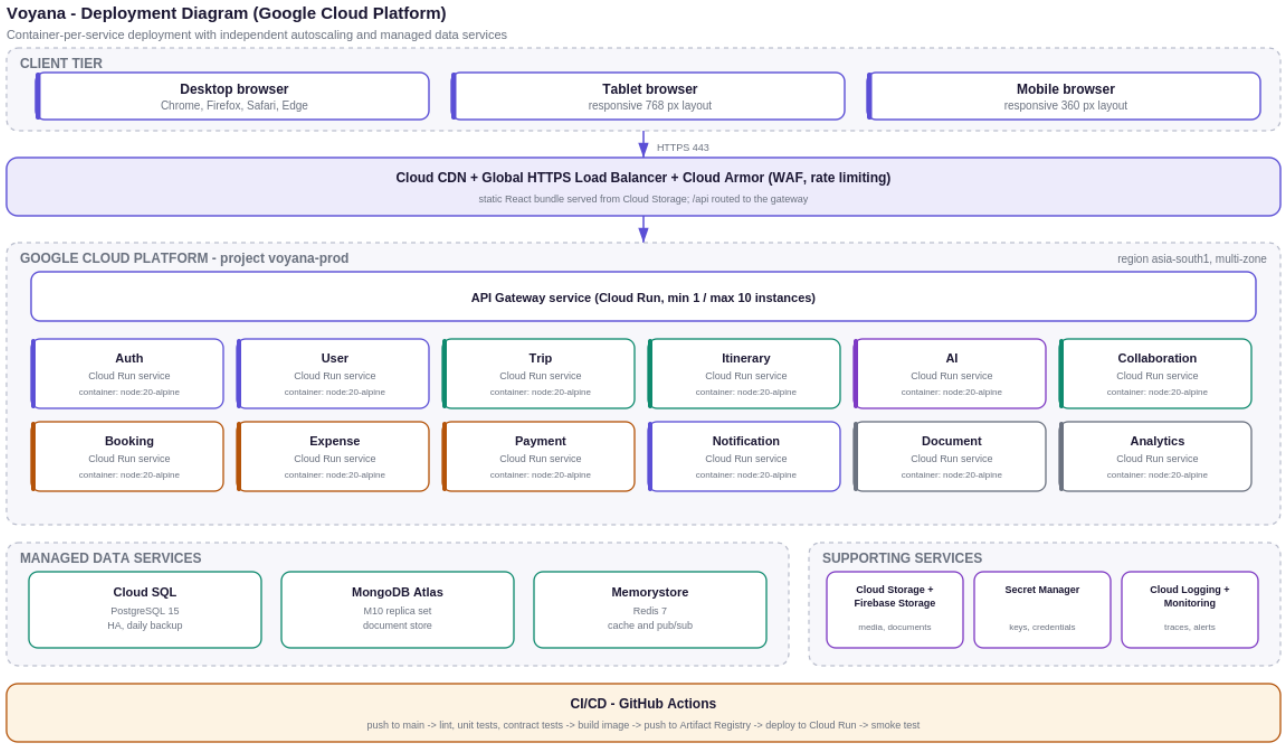
1.4 Logical Architecture Diagram



LINK TO THE DIAGRAM: https://drive.google.com/file/d/1iJUm78vfqS4gtemqk3lxQXlF_AfXYbSP/view?usp=sharing

Figure 1. Logical architecture. Presentation, gateway, service, data and integration layers.

1.5 Deployment Diagram



LINK TO THE DIAGRAM: https://drive.google.com/file/d/1a12j4dLiI3Gz6J9BQ14qjEPPKsuJiGHu/view?usp=sharing

Figure 2. Deployment view on Google Cloud Platform with per-service Cloud Run containers and managed data services.

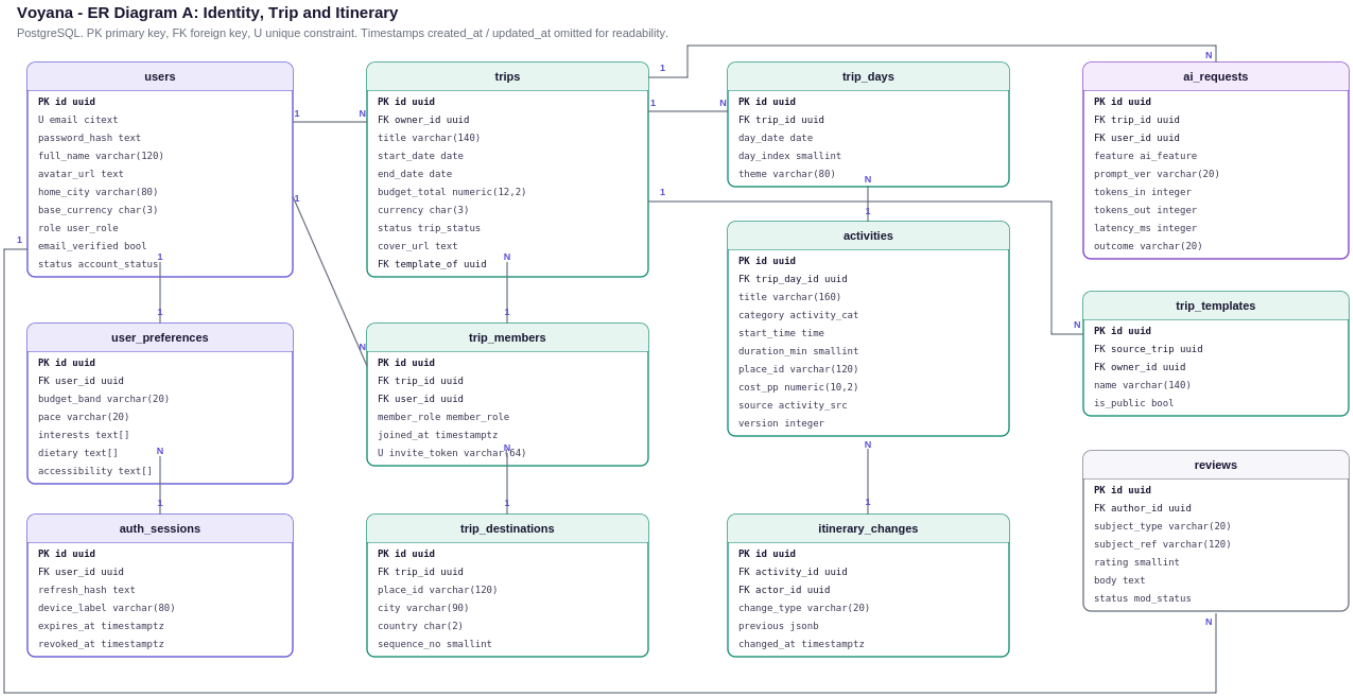
2. Database Schema

Persistence is polyglot and partitioned by service. PostgreSQL holds every entity where a transaction, a constraint or an exact numeric type matters. MongoDB holds append-heavy, schema-loose collections. Redis holds nothing that cannot be rebuilt. Because each service owns its schema, a relationship that crosses a service boundary is modelled as a stored identifier without a database-level foreign key; those relationships are drawn with dashed connectors in the diagrams that follow.

2.1 Modelling Conventions

- Primary keys are UUID v4, generated by the service rather than the database, so an identifier can be minted before a transaction opens.
- Money is stored as numeric(12,2) with a separate three-letter currency column. Floating point is not used anywhere in the schema.
- Every table carries created_at and updated_at as timestamptz; these are omitted from the diagrams for readability.
- Soft deletion is used only where an audit trail depends on the row surviving: expenses, bookings and memories. Everything else is deleted outright.
- Enumerated columns use PostgreSQL native enum types so an invalid state cannot be written even by a direct database session.
- All timestamps are stored in UTC. Trip-local presentation is resolved in the client from the destination time zone.

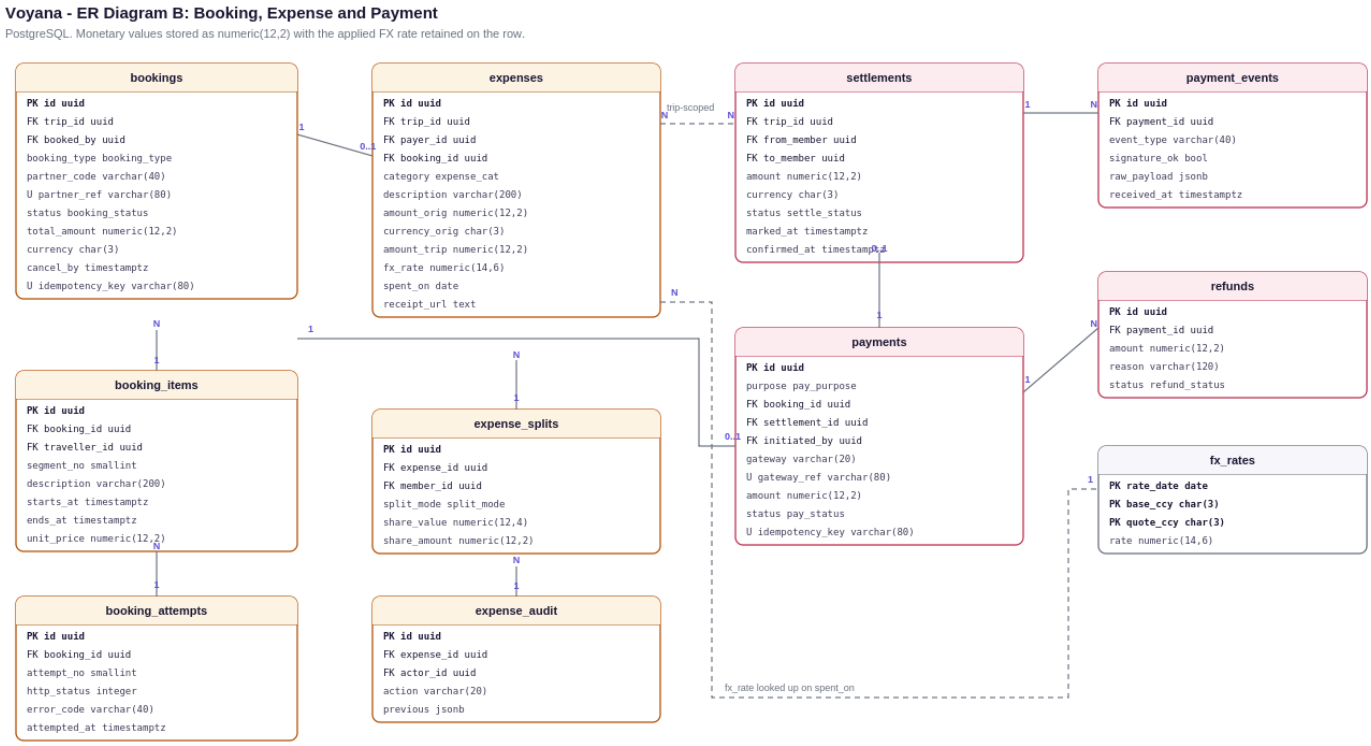
2.2 ER Diagram A - Identity, Trip and Itinerary



LINK TO THE DIAGRAM: https://drive.google.com/file/d/1n5Os3ye0c41Ala-G7jkM3N89MinM_FEy/view?usp=sharing

Figure 3. ER diagram A. Identity, trip, membership, days, activities and the itinerary change log.

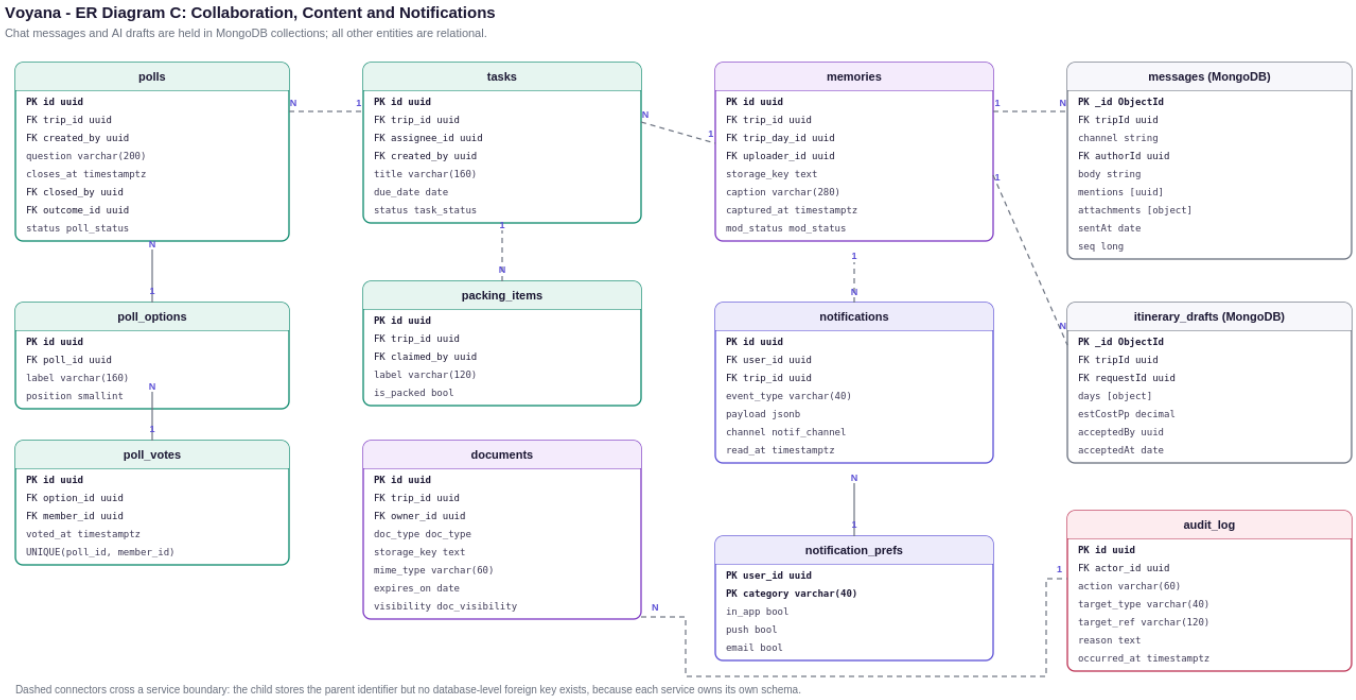
2.3 ER Diagram B - Booking, Expense and Payment



LINK TO THE DIAGRAM: https://drive.google.com/file/d/1Qam67CSg9U9Ukc4WfoIJcnIPXF4vJSWD/view?usp=sharing

Figure 4. ER diagram B. Bookings, expenses, splits, settlements, payments and refunds.

2.4 ER Diagram C - Collaboration, Content and Notifications



LINK TO THE DIAGRAM: https://drive.google.com/file/d/1qQzsBJCIkVO7kcKDMjyKDXm1rdyvDAJN/view?usp=sharing

Figure 5. ER diagram C. Polls, tasks, documents, memories, notifications and the MongoDB collections.

2.5 Table Inventory

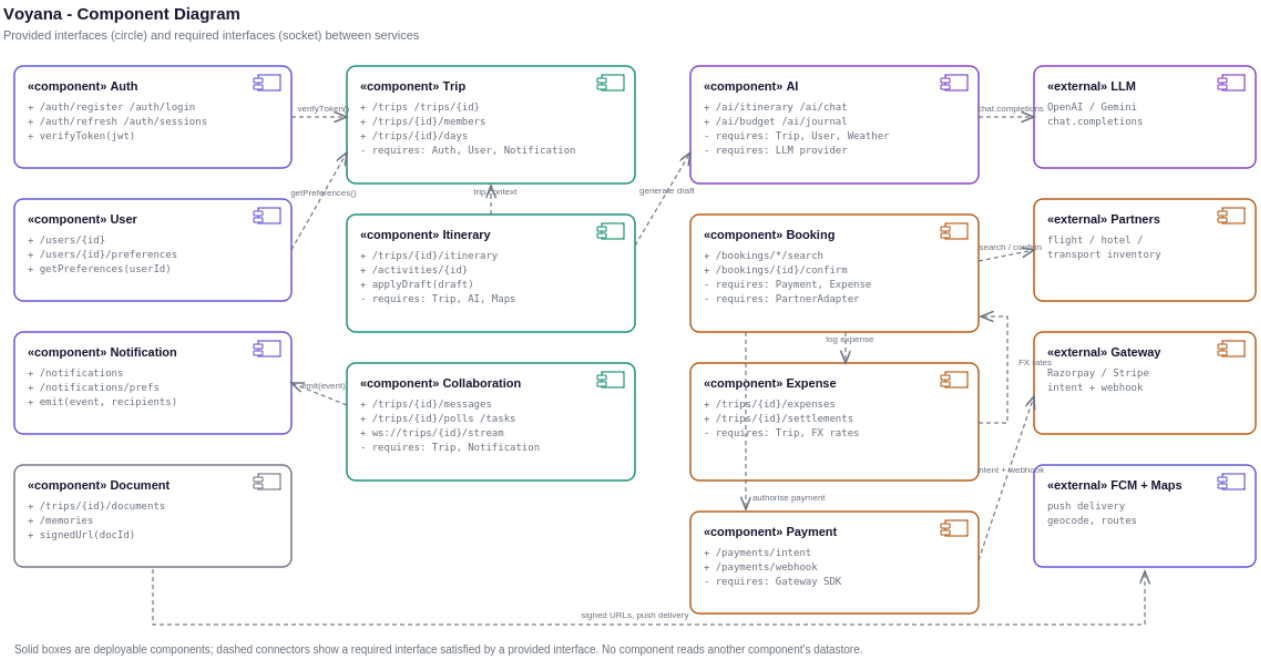
Twenty-nine relational tables and two document collections, grouped by owning service.

Owning service	Tables and collections	Store
Auth	users, auth_sessions, verification_codes	PostgreSQL
User	user_preferences, user_connections	PostgreSQL
Trip	trips, trip_members, trip_destinations, trip_days, trip_templates	PostgreSQL
Itinerary	activities, itinerary_changes	PostgreSQL
AI	ai_requests | itinerary_drafts	PostgreSQL | MongoDB
Collaboration	polls, poll_options, poll_votes, tasks, packing_items | messages	PostgreSQL | MongoDB
Booking	bookings, booking_items, booking_attempts	PostgreSQL
Expense	expenses, expense_splits, expense_audit, fx_rates	PostgreSQL
Payment	payments, payment_events, refunds, settlements	PostgreSQL
Notification	notifications, notification_prefs	PostgreSQL
Document	documents, document_access_log, memories	PostgreSQL
Analytics	audit_log, reviews	PostgreSQL

2.6 Indexes, Constraints and Integrity Rules

Object	Rule	Purpose
users.email	UNIQUE on citext	Case-insensitive uniqueness without a functional index at query time.
trip_members	UNIQUE (trip_id, user_id)	A user cannot join the same trip twice.
poll_votes	UNIQUE (poll_id, member_id)	One vote per member; a re-vote is an UPDATE, never an INSERT.
activities	INDEX (trip_day_id, start_time)	Day view is the hottest read path in the product.
activities.version	Incremented on every write; checked against If-Match	Implements optimistic locking for FR-TRP-08.
expenses	INDEX (trip_id, spent_on DESC)	Expense log and budget roll-up both scan by trip and date.
expense_splits	CHECK sum(share_amount) = expenses.amount_trip, enforced in a trigger	A split that does not reconcile can never be persisted.
bookings.idempotency_key	UNIQUE	A retried confirm returns the original booking instead of creating a second one.
payments.gateway_ref	UNIQUE	A duplicated webhook cannot create a second payment row.
settlements	CHECK from_member <> to_member	A member cannot settle with themselves.
documents	INDEX (trip_id, owner_id, visibility)	Vault listing must filter by visibility on every read.
fx_rates	PRIMARY KEY (rate_date, base_ccy, quote_ccy)	One rate per currency pair per day; historical values are immutable.
messages (Mongo)	INDEX { tripId: 1, seq: 1 }	Ordered replay after a socket reconnect without duplication.
audit_log	Append only; no UPDATE or DELETE grant	A moderation or administrative action cannot be erased.

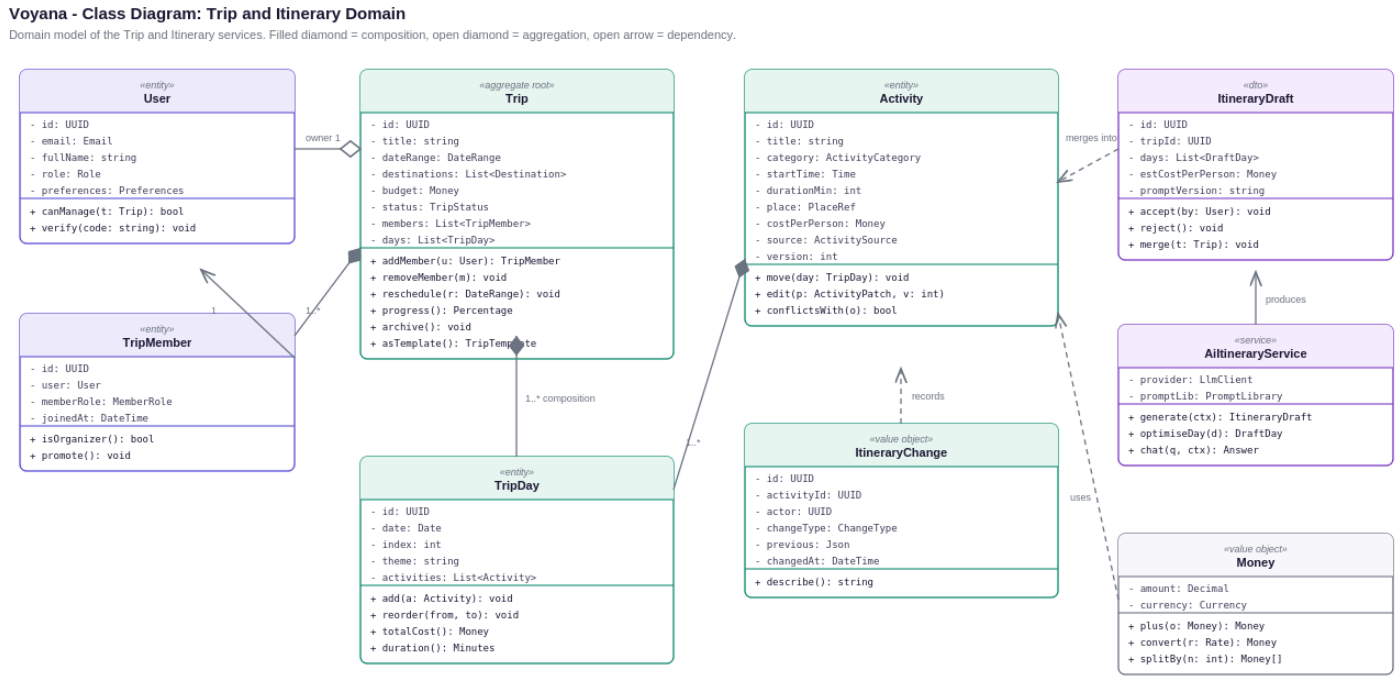
3. Component and Class Diagrams



LINK TO THE DIAGRAM: https://drive.google.com/file/d/1OXejIvIQvP8O_N43WWOjuac_iZo9Wvtn/view?usp=sharing

Figure 6. Component diagram. Provided and required interfaces between the twelve services and the external systems.

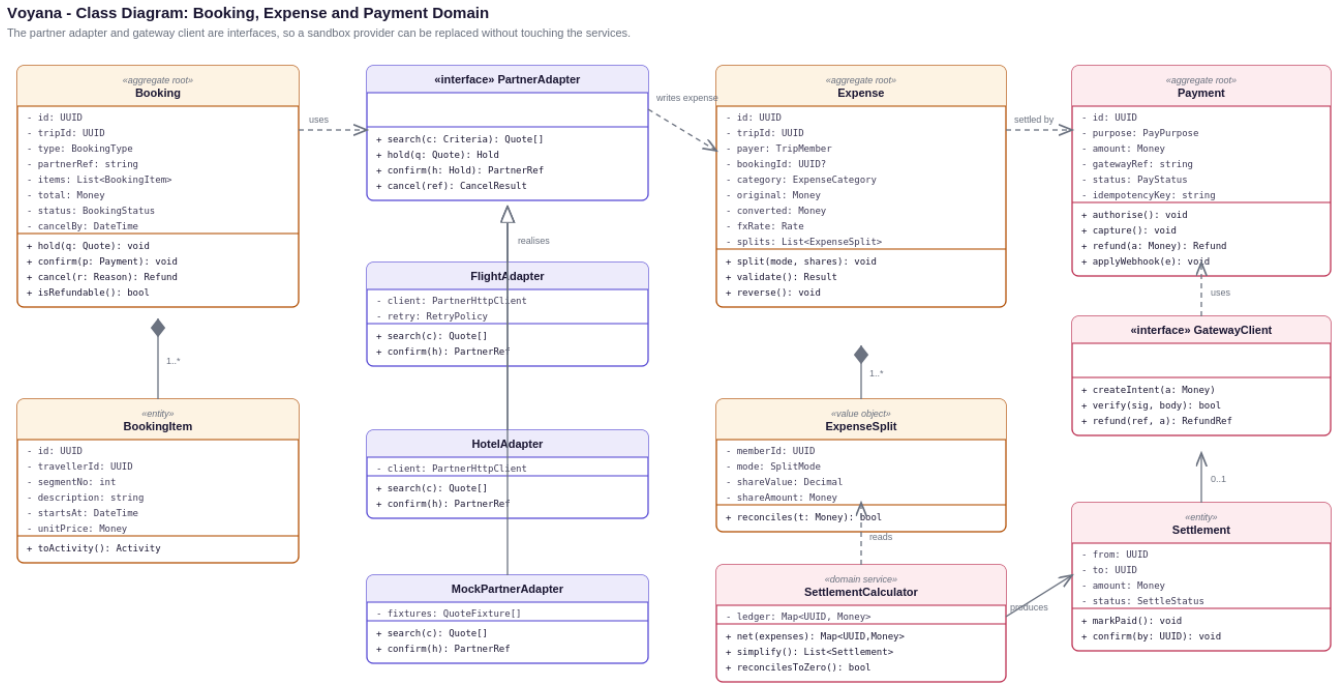
3.1 Class Diagram - Trip and Itinerary Domain



LINK TO THE DIAGRAM: https://drive.google.com/file/d/1XiqMjSppmZMHVI7JtPUlNmBAEYZTudJA/view?usp=sharing

Figure 7. Class diagram for the Trip and Itinerary domain, showing composition of days and activities and the draft merge path.

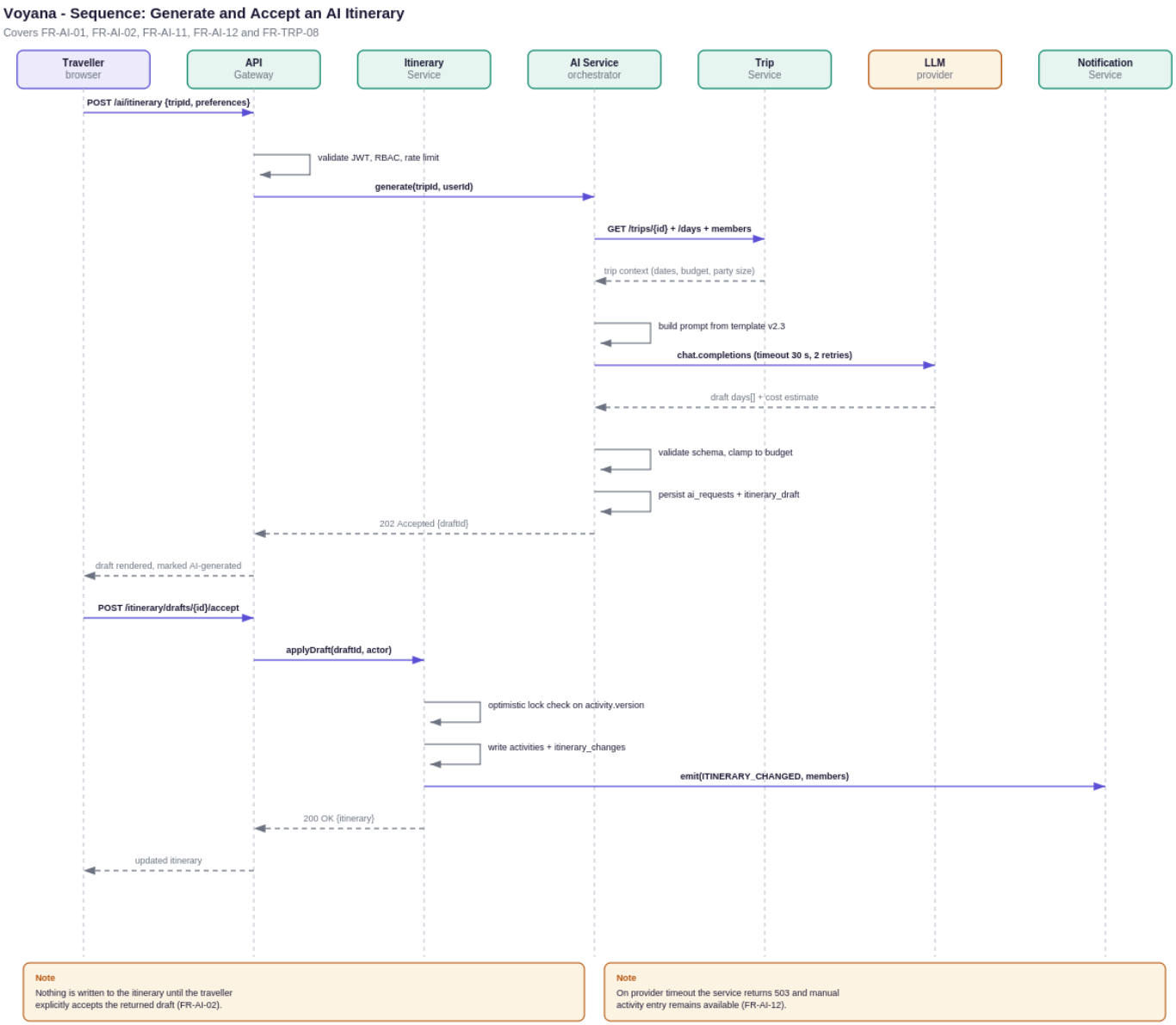
3.2 Class Diagram- Booking, Expense and Payment Domain



LINK TO THE DIAGRAM: https://drive.google.com/file/d/100ZeyjRMAMeaRFTukoT3Y4vxlVqs5upV/view?usp=sharing

Figure 8. Class diagram for the Booking, Expense and Payment domain, with adapter and gateway interfaces and their realisations.

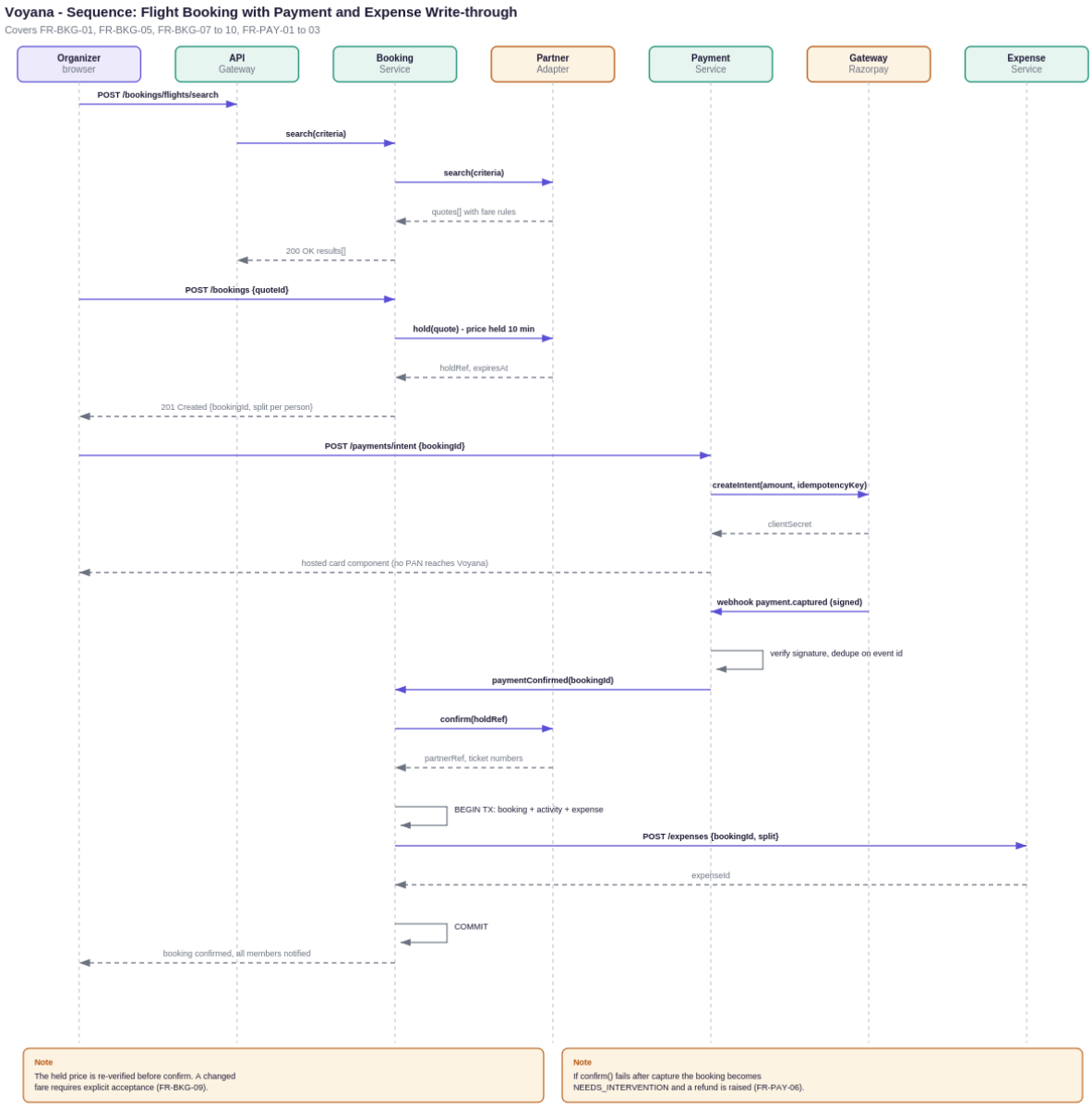
3.3 Sequence Diagram - Generate and Accept an AI Itinerary



LINK TO THE DIAGRAM: https://drive.google.com/file/d/1307UvYUMH2eAue2tBtsyj3CYokiP8cwT/view?usp=sharing

Figure 9. Sequence: AI itinerary generation and acceptance. Nothing is written until the traveller accepts the draft.

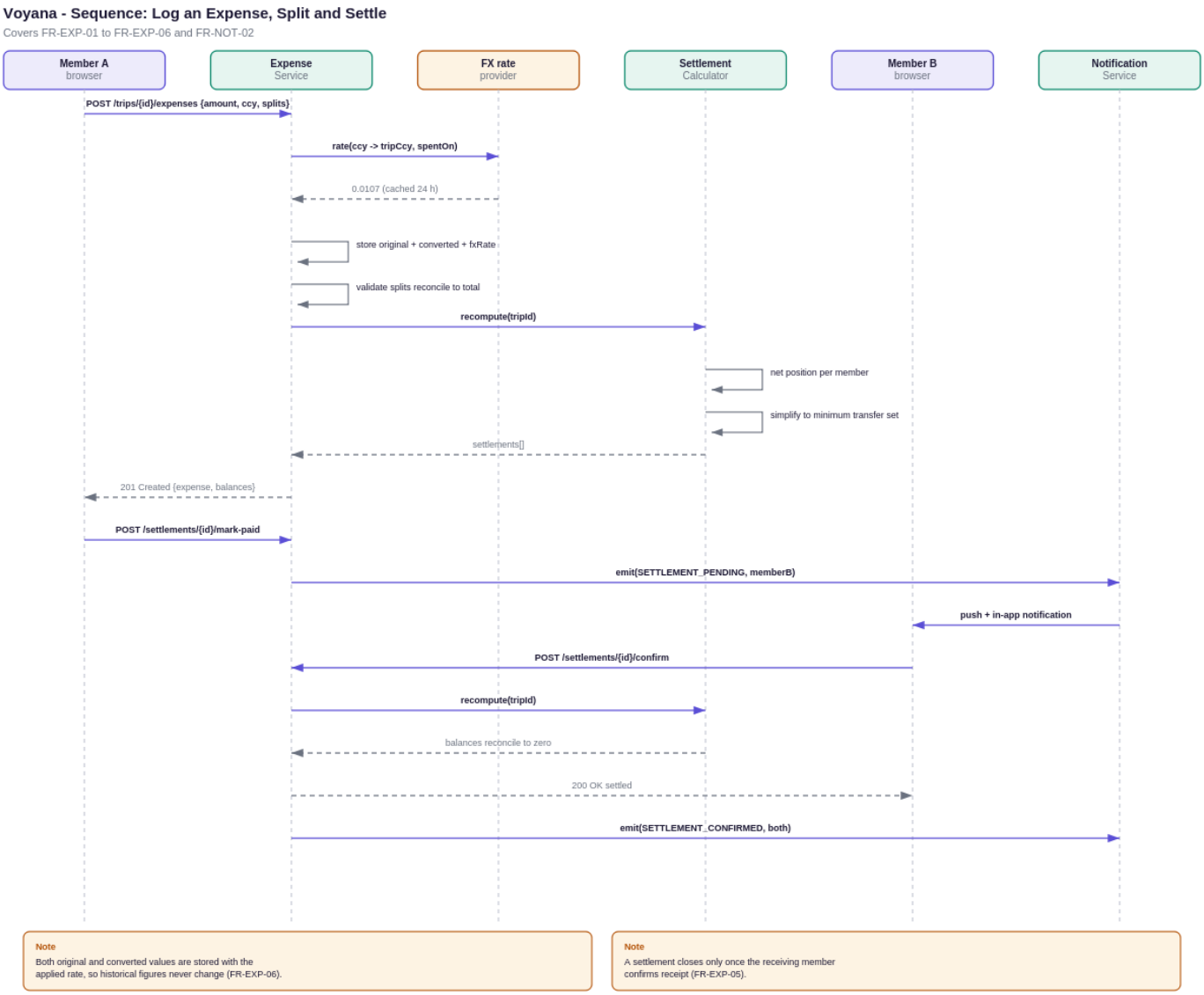
3.4 Sequence Diagram - Booking with Payment and Expense Write-through



LINK TO THE DIAGRAM: https://drive.google.com/file/d/1-N5vmhgcwXIUuK2YqfsHTHMrKQ7gX0Xi/view?usp=sharing

Figure 10. Sequence: flight booking, payment capture and the atomic write of booking, activity and expense.

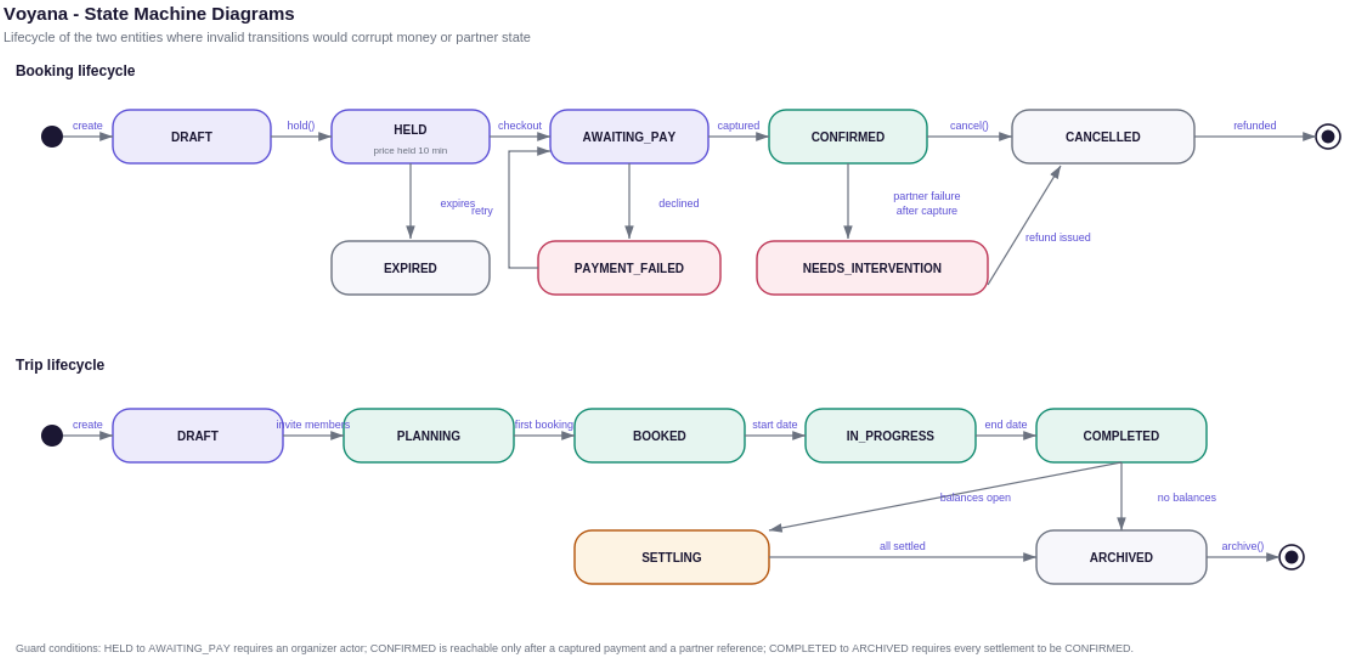
3.5 Sequence Diagram - Log an Expense, Split and Settle



LINK TO THE DIAGRAM: https://drive.google.com/file/d/1Ei7PPFtc5LOInLlH_4WG3HuyxsL-rg9U/view?usp=sharing

Figure 11. Sequence: expense logging, currency conversion, settlement simplification and two-sided confirmation.

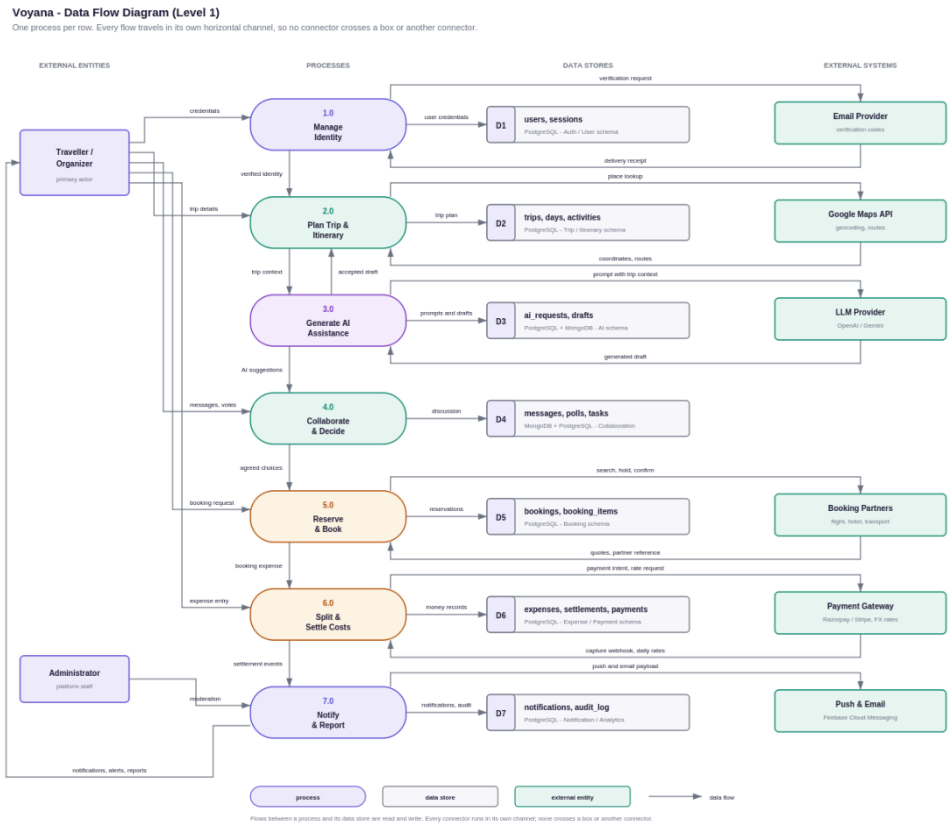
3.6 State Machine Diagrams



LINK TO THE DIAGRAM: https://drive.google.com/file/d/1TF4Gkrsqr9B3bkilTxFgwpgXUHPY11_b/view?usp=sharing

Figure 12. State machines for the booking and trip lifecycles, including the failure states that protect money and partner state.

3.7 Data Flow Diagram (Level 1)



LINK TO THE DIAGRAM: https://drive.google.com/file/d/1pajwqxlv1pfOyHDXioUPXHJGDYHk4T13/view?usp=sharing

Figure 13. Level 1 data flow diagram showing external entities, the seven top-level processes and the four data stores.

4. REST API Contracts

The complete machine-readable contract is delivered as 2026-08-22_Voyana_OpenAPI_v1.0.yaml, an OpenAPI 3.0.3 document covering 49 paths, 59 operations and 55 schemas across the twelve services. It can be opened directly in Swagger Editor or imported into Postman. This section states the conventions that govern the whole contract and reproduces the operations that carry the most design weight.

4.1 Contract Conventions

Convention	Rule
Base path	https://api.voyana.app/api/v1. A breaking change requires /api/v2; additive fields never bump the version.
Authentication	Bearer JWT in the Authorization header. Access tokens last 15 minutes; refresh tokens rotate on every use. No credential is ever accepted in a query string.
Authorisation	The gateway performs a coarse role check; every service repeats the trip-membership check against its own data. A non-member receives 403 with no body content from the trip.
Correlation	Every request carries X-Correlation-Id, generated at the gateway when absent and propagated unchanged to every downstream call and log line.
Idempotency	Booking, payment and expense writes require an Idempotency-Key header. A repeated key returns the original result rather than creating a duplicate.
Concurrency	Itinerary writes require If-Match carrying the version token from the last read. A stale token returns 409 with both the attempted and the current value.

Convention	Rule
Pagination	Cursor based. Collection responses return items, nextCursor and hasMore. No endpoint returns an unbounded collection.
Money	Sent as an object of amount (decimal string) and currency (ISO 4217). Floating point is never used on the wire.
Errors	A single envelope for every service: code, message, correlationId and an optional field-level details array.
Documentation	Each service publishes its own slice of the contract at /docs; the gateway serves the merged document.

4.2 Operation Inventory

Service	Operations	Representative paths
Authentication	8	POST /auth/register, /auth/verify, /auth/login, /auth/oauth/{provider}, /auth/refresh; GET, DELETE /auth/sessions
Users	4	GET, PATCH /users/me; PUT /users/me/preferences; POST /users/me/export
Trips	8	GET, POST /trips; GET, PATCH, DELETE /trips/{tripId}; GET, POST /trips/{tripId}/members
Itinerary	5	GET /trips/{tripId}/itinerary; POST /trips/{tripId}/days/{dayId}/activities; PATCH, DELETE /activities/{id}
AI	5	POST /ai/itinerary, /ai/itinerary/drafts/{id}/accept, /ai/chat, /ai/budget, /ai/journal
Collaboration	7	GET, POST /trips/{tripId}/messages; POST /trips/{tripId}/polls; PUT /polls/{id}/votes; POST /polls/{id}/close
Bookings	7	POST /bookings/flights/search, /bookings/hotels/search, /bookings, /bookings/{id}/confirm; DELETE /bookings/{id}
Expenses	8	GET, POST /trips/{tripId}/expenses; PATCH, DELETE /expenses/{id}; GET /trips/{tripId}/settlements
Payments	3	POST /payments/intent, /payments/webhook, /payments/{id}/refund
Notifications	2	GET /notifications; PUT /notifications/preferences
Documents	4	POST /trips/{tripId}/documents, /trips/{tripId}/memories; GET /documents/{id}/url
Admin	3	GET /admin/moderation, /admin/health; POST /admin/moderation/{id}/decision

4.3 Representative Contracts

POST /trips - create a trip

Creates the trip, makes the caller its organizer and generates one day container per date in the range.

```
Request
{
  "title": "Bali Getaway",
  "destinations": [ { "placeId": "ChIJ...", "city": "Ubud", "country": "ID" } ],
  "startDate": "2026-12-12",
  "endDate":   "2026-12-19",
  "budget":    { "amount": "60000.00", "currency": "INR" }
}
```

```
201 Created
{
  "id": "9f1c...", "title": "Bali Getaway", "status": "planning",
  "startDate": "2026-12-12", "endDate": "2026-12-19",
  "budget": { "amount": "60000.00", "currency": "INR" },
  "spent":  { "amount": "0.00", "currency": "INR" },
  "memberCount": 1, "progressPct": 0
}
```

Status	Meaning
201	Trip created; day containers generated.
422	End date precedes start date, or the budget currency is not ISO 4217.
401	Missing or expired access token.

PATCH /activities/{activityId} - optimistic locking

The only write in the product where two members routinely collide. The client must echo the version token it last read.

```
PATCH /api/v1/activities/3b7e...   If-Match: "7"
{ "startTime": "10:30", "durationMin": 90 }
```

```
409 Conflict - a concurrent edit won
{
  "code": "STALE_WRITE",
  "attempted": { "startTime": "10:30", "version": 7 },
  "current":   { "startTime": "09:45", "version": 8 }
}
```

POST /bookings/{bookingId}/confirm - atomic confirmation

Organizer only. On success the booking, the itinerary activity and the linked expense are written in a single transaction; on partner failure after capture the booking moves to needs_intervention and a refund is raised.

```
Headers: Authorization: Bearer <jwt>   Idempotency-Key: 9d2f-...
{ "paymentId": "c41a...", "acceptNewPrice": false }
```

```
409 Conflict - held price changed
{ "code": "PRICE_CHANGED",
  "heldPrice": { "amount": "138250.00", "currency": "INR" },
  "newPrice":  { "amount": "141900.00", "currency": "INR" } }
```

GET /trips/{tripId}/settlements - simplified transfer set

Returns net positions and the minimum transfer set. The reconciled flag is asserted by the automated test suite on every seeded dataset.

```
200 OK
{
  "netPositions": [
    { "memberId": "a1...", "balance": { "amount": "-8400.00", "currency": "INR" } },
    { "memberId": "b2...", "balance": { "amount": "14000.00", "currency": "INR" } }
  ],
  "transfers": [
    { "id": "t1", "fromMember": "a1...", "toMember": "b2...",
      "amount": { "amount": "8400.00", "currency": "INR" }, "status": "pending" }
  ],
  "reconciled": true
}
```

4.4 Error Model and Status Code Usage

Code	Used for	Example code value
400	Malformed body or failed webhook signature verification.	SIGNATURE_INVALID
401	No credential, expired token, or invalid refresh token.	TOKEN_EXPIRED
403	Authenticated but not a member of the trip, or not an organizer.	NOT_TRIP_MEMBER
404	Resource absent, or hidden because the caller may not know it exists.	TRIP_NOT_FOUND

Code	Used for	Example code value
409	State conflict: stale write, closed poll, changed price, unsettled balance.	STALE_WRITE, PRICE_CHANGED
410	Verification code or invitation link expired.	CODE_EXPIRED
413	Upload exceeds the 10 MB document limit.	FILE_TOO_LARGE
422	Schema valid but business rule violated.	SPLIT_DOES_NOT_RECONCILE
423	Account locked after five consecutive failed sign-ins.	ACCOUNT_LOCKED
429	Rate limit exceeded at the gateway or by the LLM provider.	RATE_LIMITED
502	Partner API failed after two retries; no partial record created.	PARTNER_UNAVAILABLE
503	AI provider unavailable; manual entry remains available.	AI_PROVIDER_DOWN
504	Partner or provider timeout.	UPSTREAM_TIMEOUT

4.5 Security Design Applied to the Contract

- Card data never reaches a Voyana endpoint. /payments/intent returns a client secret and capture happens in the gateway-hosted component.
- /payments/webhook is the only unauthenticated write. It is protected by signature verification, an origin allow-list and idempotent event handling.
- Document and memory bytes never transit the API. The service issues a signed URL valid for at most 15 minutes and logs every issuance.
- Trip-scoped endpoints return 403 with an empty body rather than 404, except where disclosing existence would itself leak information.
- Every AI response carries aiGenerated true, so the client can label it as required by the interface requirements.

5. Wireframes and UI/UX Mockups (final)

5.1. Landing Page

Unauthenticated entry point. The interactive globe and destination search are the primary affordance; the itinerary preview card acts as proof of the value proposition rather than a description of it.

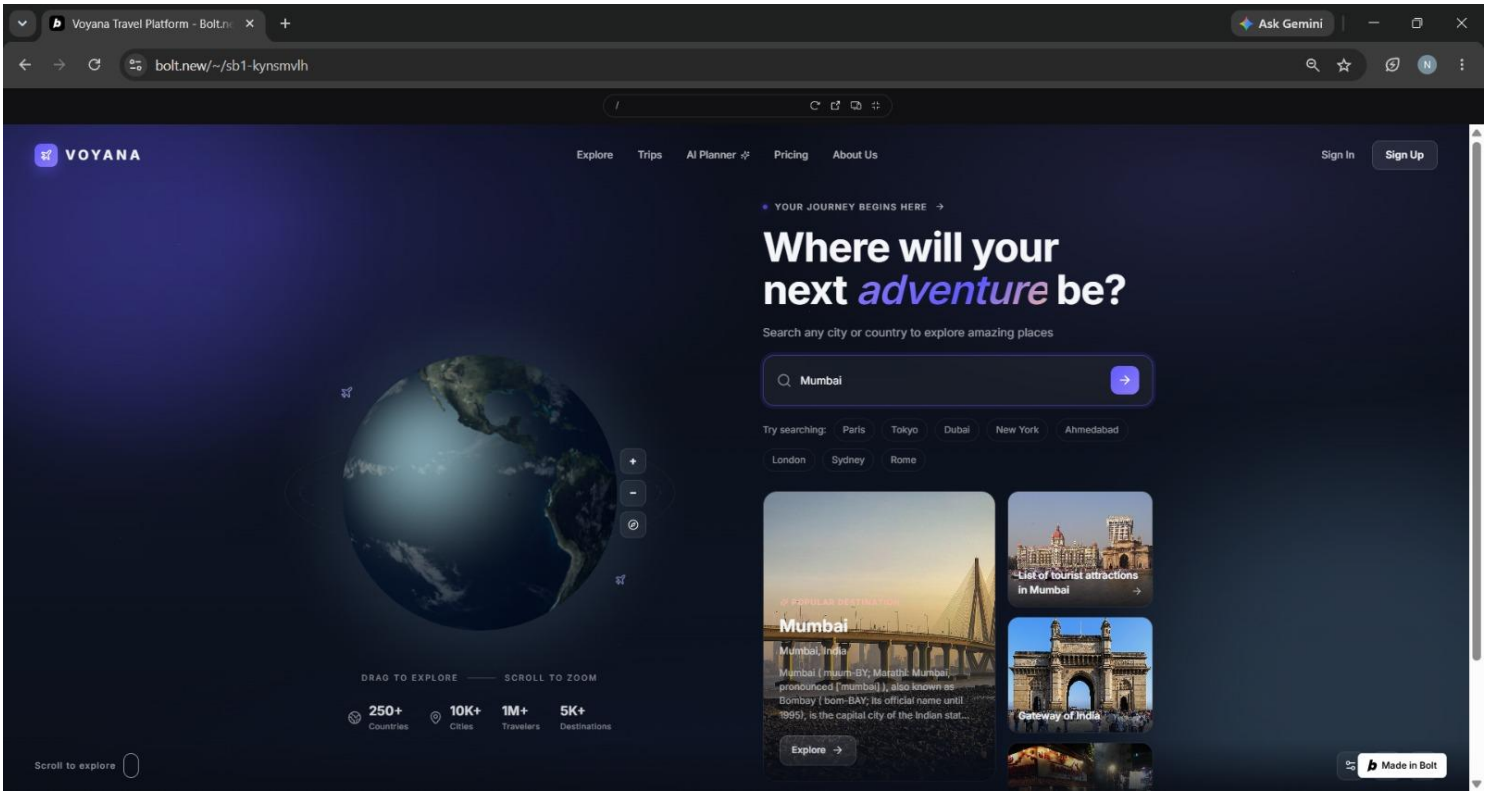


Figure 14. Public Landing Page - final mockup.

Design elements	Services and endpoints behind the screen
Globe canvas, destination search with suggestion chips, result cards, statistics strip, primary sign-up action.	Search resolves through the Trip Service place lookup, which proxies Google Maps geocoding. No authenticated call is made until sign-up.

5.2 Registration

Federated sign-in is presented first and the email path second. Password strength is live, the terms checkbox is unticked by default, and the verification-code expiry is stated before submission rather than after.

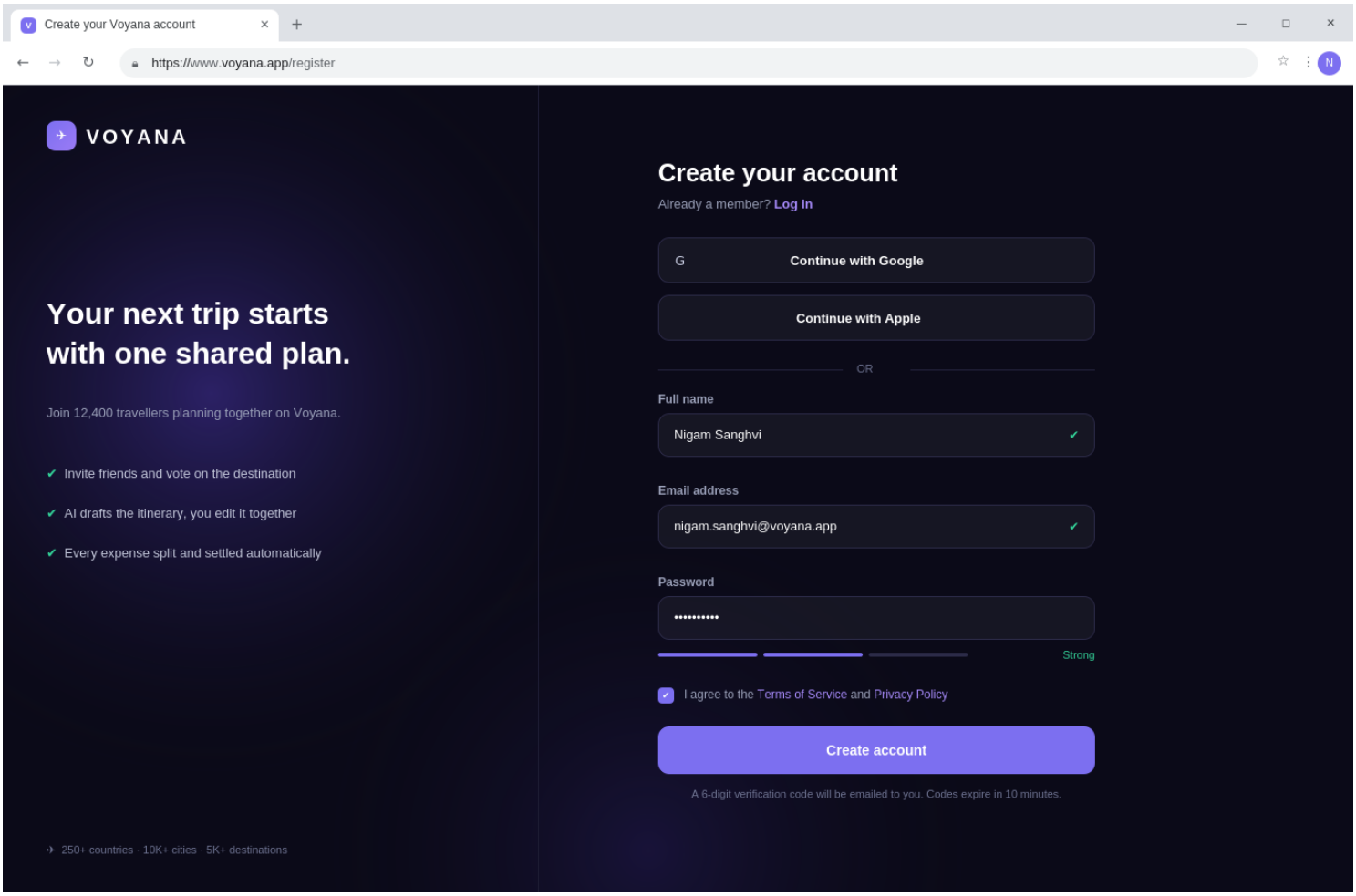


Figure 15. Registration - final mockup.

Design elements	Services and endpoints behind the screen
Two-panel split, OAuth buttons, labelled fields with inline validation, three-segment strength meter, explicit consent checkbox.	POST /auth/register, POST /auth/oauth/{provider}, POST /auth/verify. Errors render against the failing field using the details array of the error envelope.

5.3 Trip Dashboard

The authenticated home. Trip cards carry status, dates, membership and planning progress. The AI suggestion panel is advisory only: every card proposes a change and requires an explicit action.

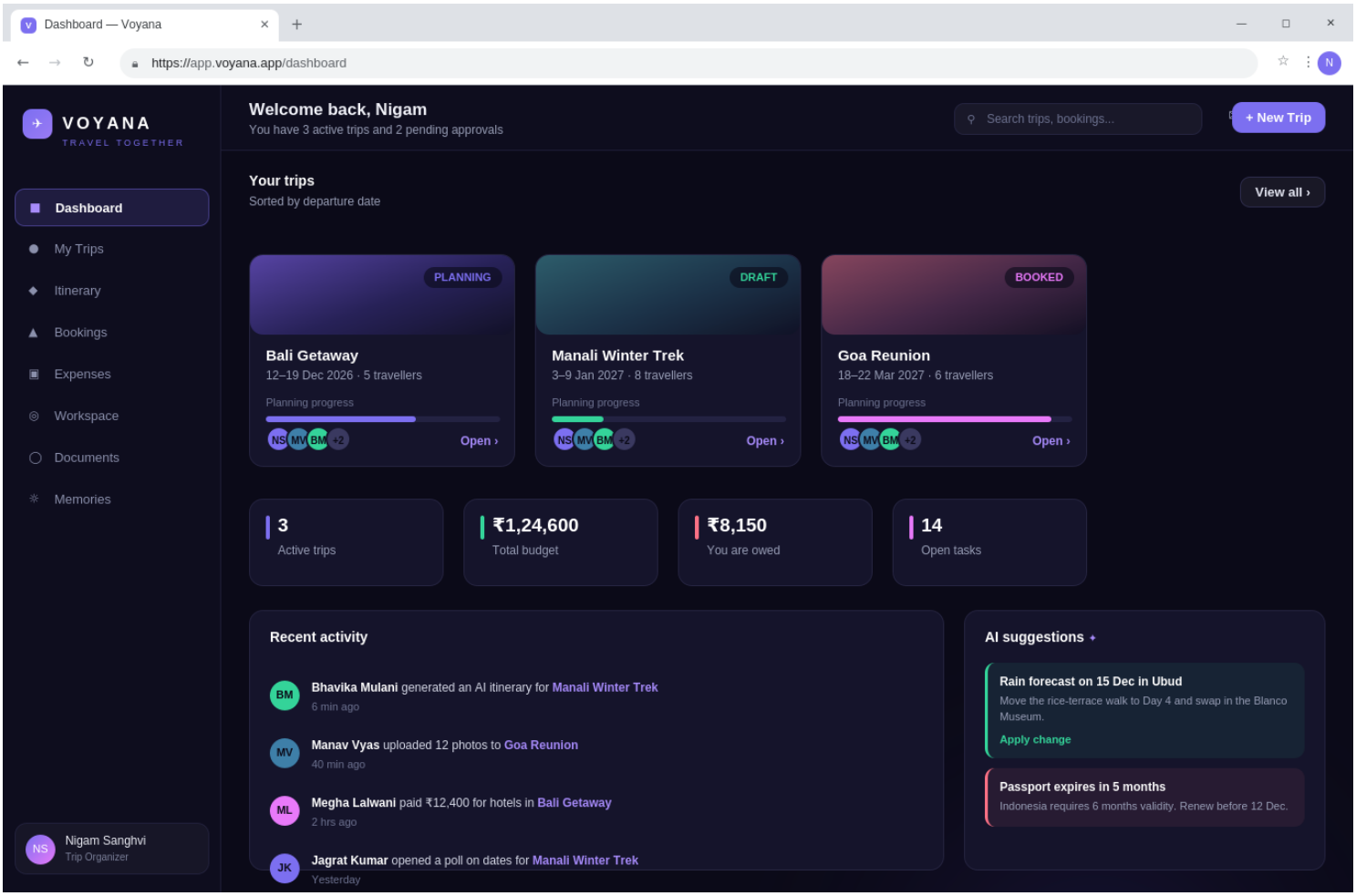


Figure 16. Trip Dashboard - final mockup.

Design elements	Services and endpoints behind the screen
Fixed sidebar, trip cards with progress and avatar stack, four-metric strip, activity feed, advisory AI panel.	GET /trips, GET /trips/{id}/settlements for the balance metric, GET /notifications for the feed, POST /ai/budget for the advisory cards.

5.4 Itinerary Builder with AI Assistant

The core working screen. Day selector, activity list and assistant occupy one view so that a conversational instruction and its result are visible together. The assistant states the cost consequence of every change it proposes.

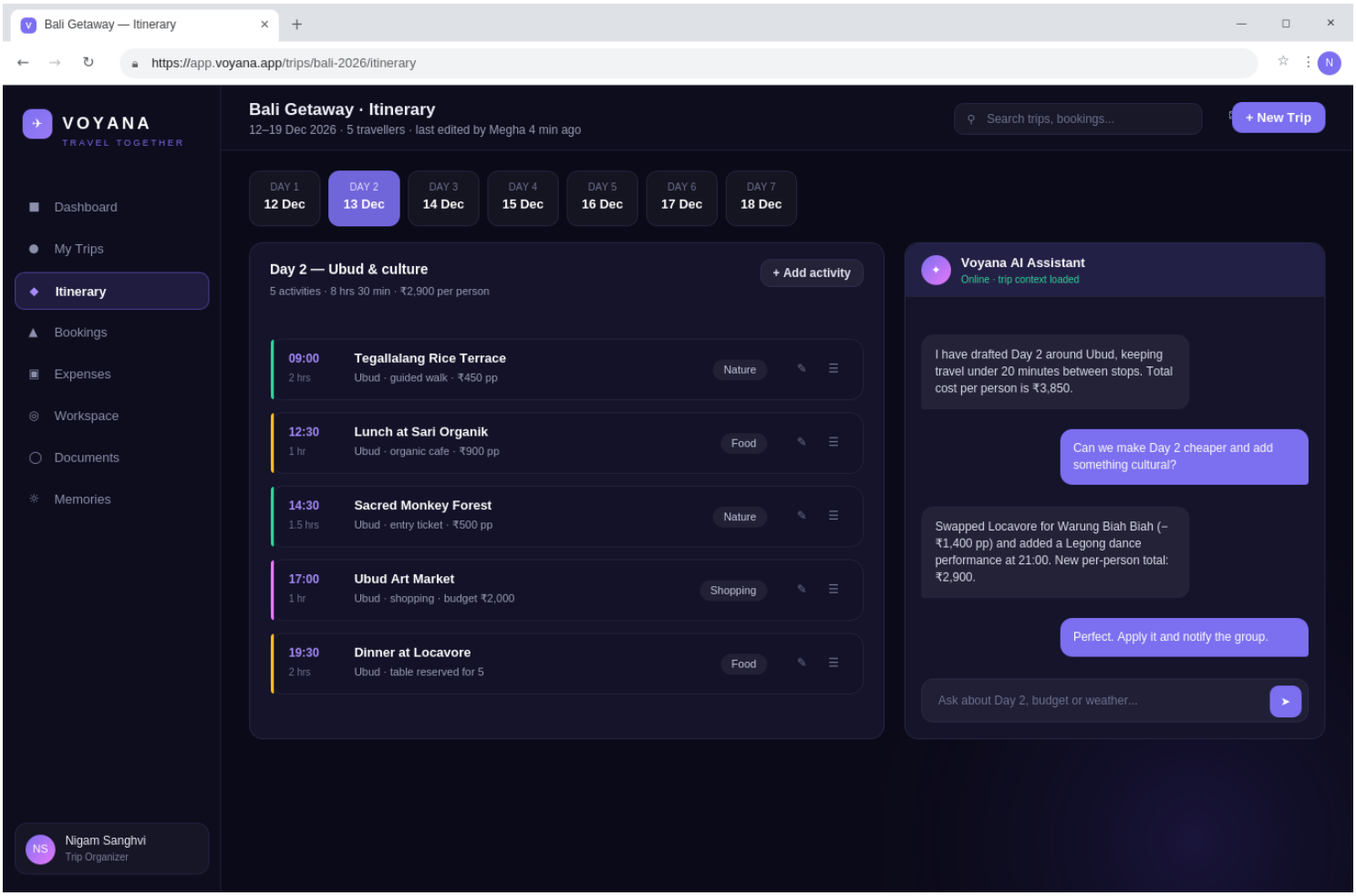


Figure 17. Itinerary Builder with AI Assistant - final mockup.

Design elements	Services and endpoints behind the screen
Day chip selector, activity rows with coloured category rail and drag handles, assistant panel with AI-marked bubbles.	GET /trips/{id}/itinerary with ETag, PATCH /activities/{id} with If-Match, POST /ai/chat, POST /ai/itinerary/drafts/{id}/accept.

5.5 Group Workspace

Chat, polls and tasks in one view, because separating them is what fragments group decisions in the tools Voyana replaces. The poll panel shows live tallies, a deadline and a single action that promotes the winner into the itinerary.

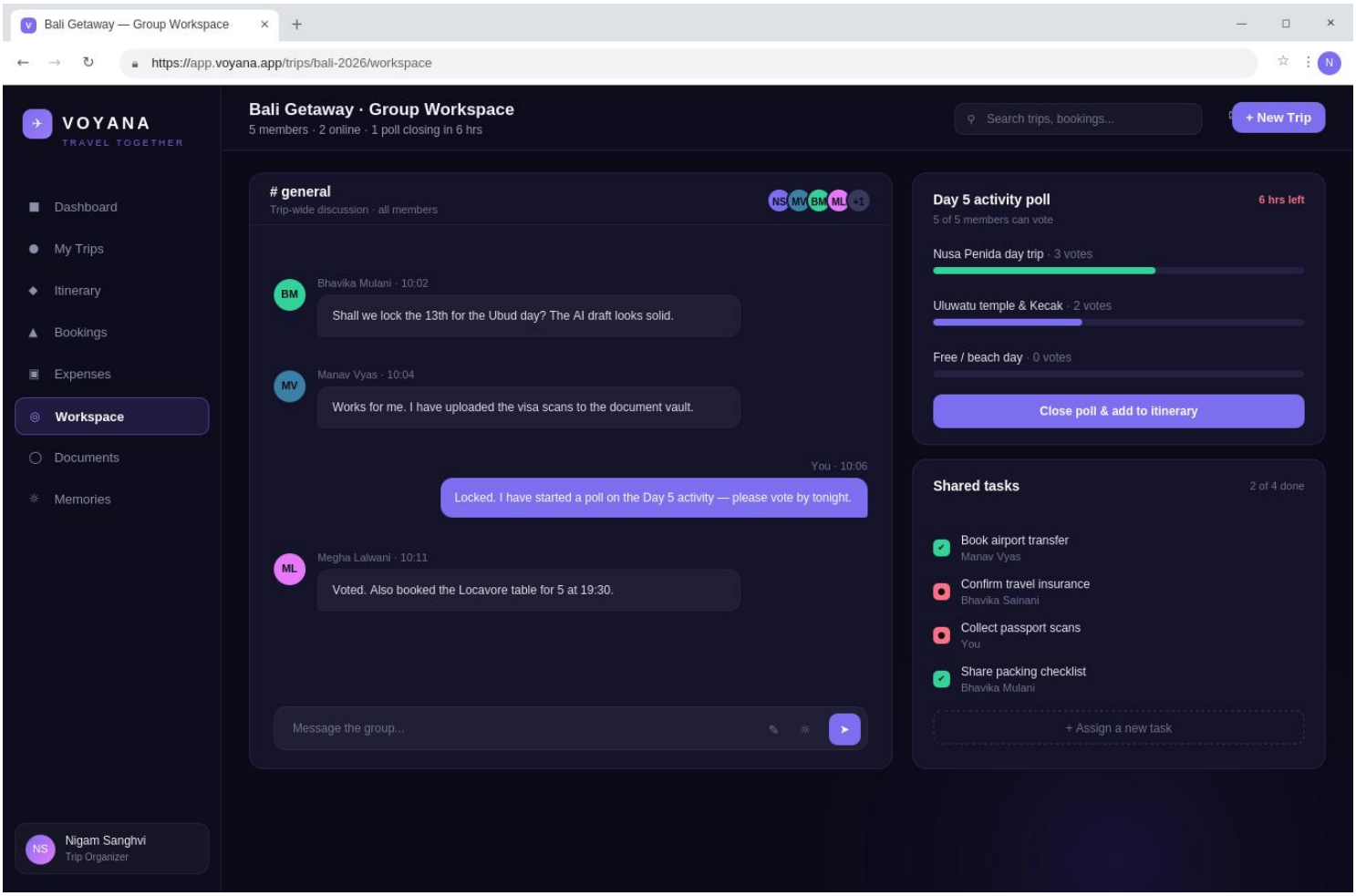


Figure 18. Group Workspace - final mockup.

Design elements	Services and endpoints behind the screen
Channel header with presence avatars, message list, composer, poll card with tally bars, shared task list.	WebSocket ws://trips/{id}/stream for delivery, GET and POST /trips/{id}/messages, PUT /polls/{id}/votes, POST /polls/{id}/close.

5.6 Expenses and Settlements

Budget consumption sits above the log so the consequence is visible before the detail. The settlement panel presents the simplified transfer set rather than the raw obligation graph.

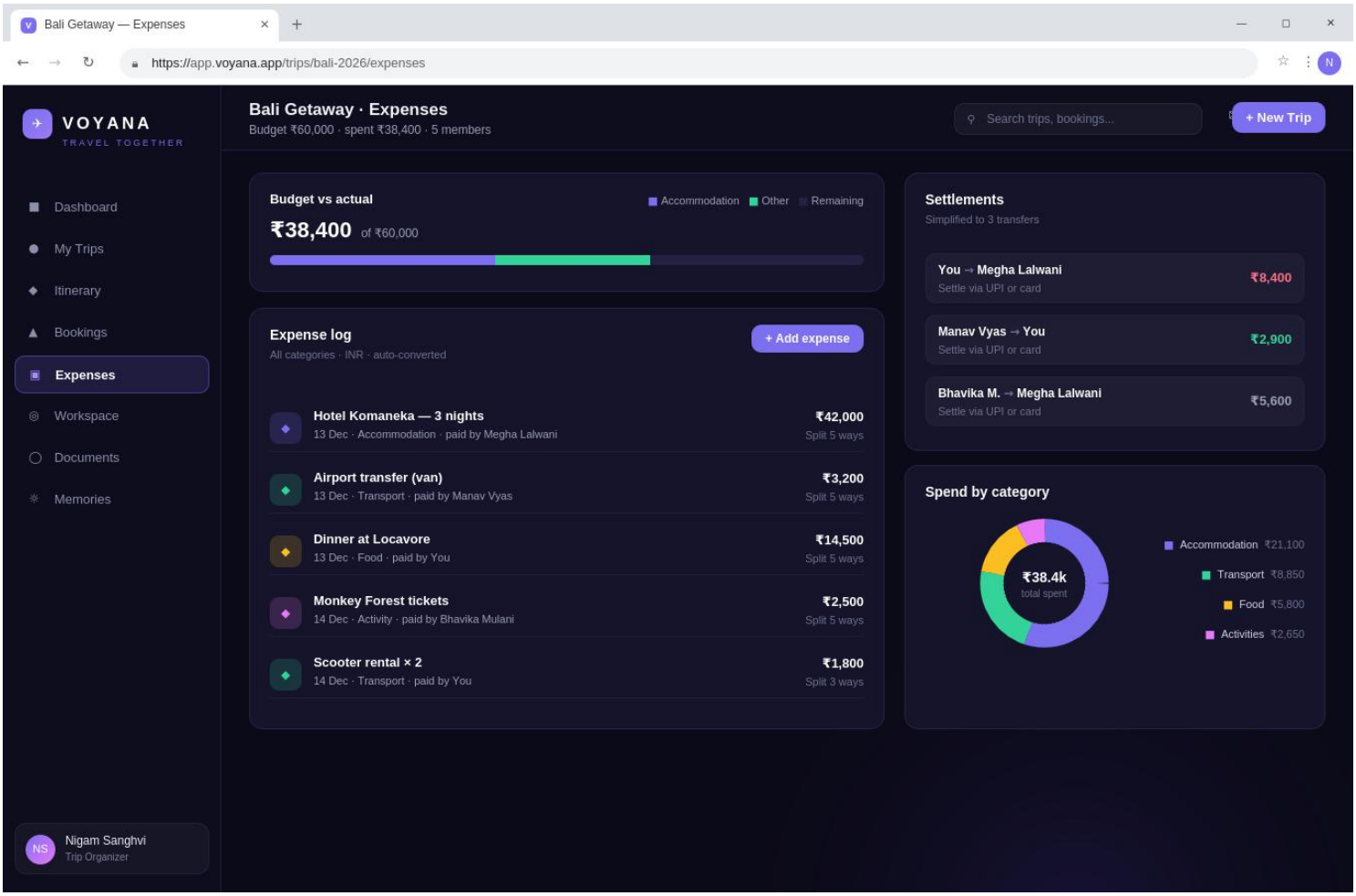


Figure 19. Expenses and Settlements - final mockup.

Design elements	Services and endpoints behind the screen
Budget bar with category segmentation, expense rows showing payer and split, settlement cards, category donut.	GET and POST /trips/{id}/expenses, GET /trips/{id}/settlements, POST /settlements/{id}/mark-paid and /confirm.

5.7 Unified Booking

Search criteria, results and the running summary occupy one view. The summary states the per-person split before payment, which is the point at which a group booking usually stalls.

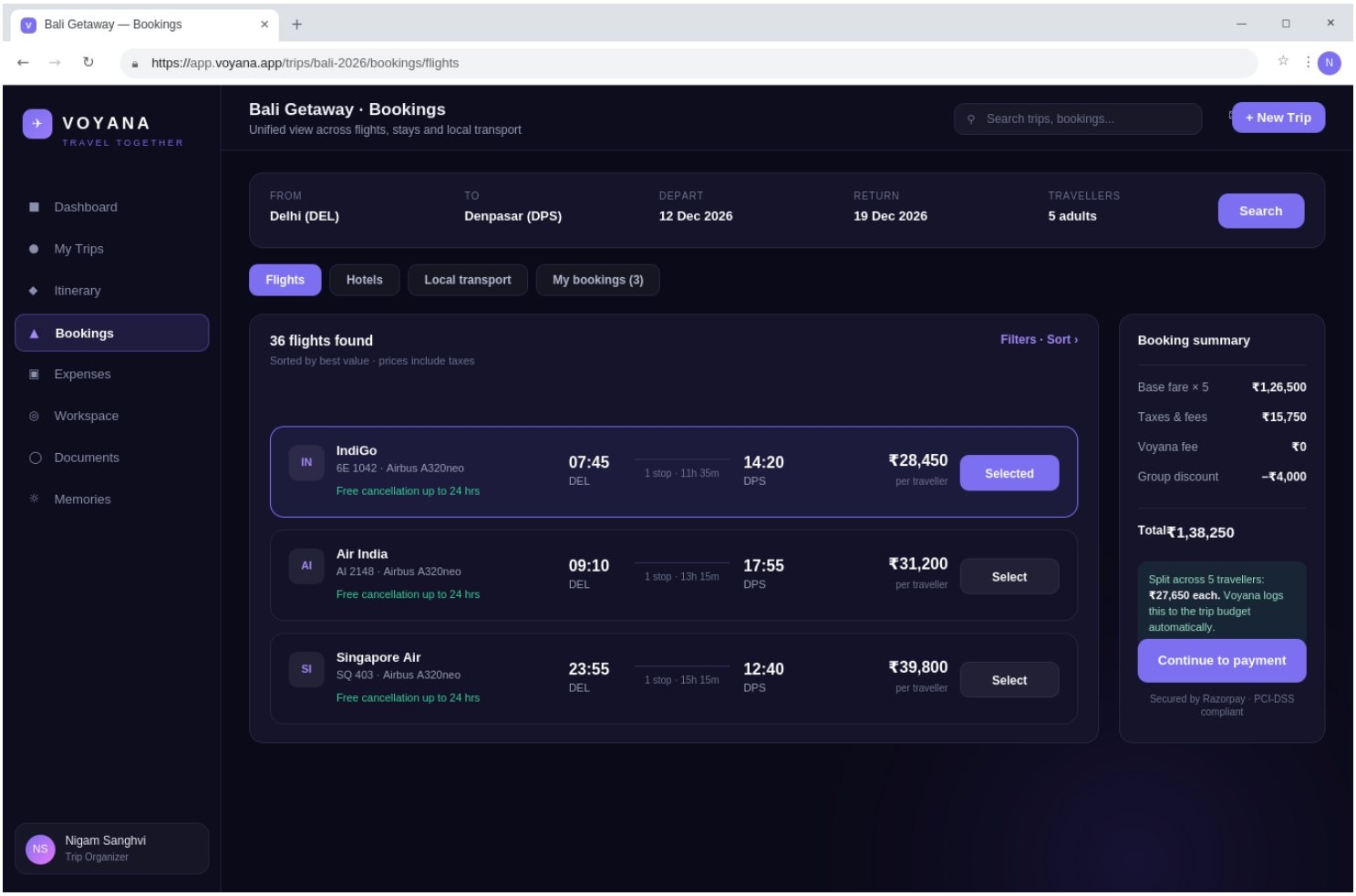


Figure 20. Unified Booking - final mockup.

Design elements	Services and endpoints behind the screen
Criteria bar, product tabs, result rows with fare rules and hold state, sticky booking summary with per-person split.	POST /bookings/flights/search, POST /bookings (hold), POST /payments/intent, POST /bookings/{id}/confirm with Idempotency-Key.

5.8 Administration and Analytics Console

The administrator view. It combines platform metrics, a service health panel that reflects the twelve-service architecture, and a moderation queue where every action records a reason.

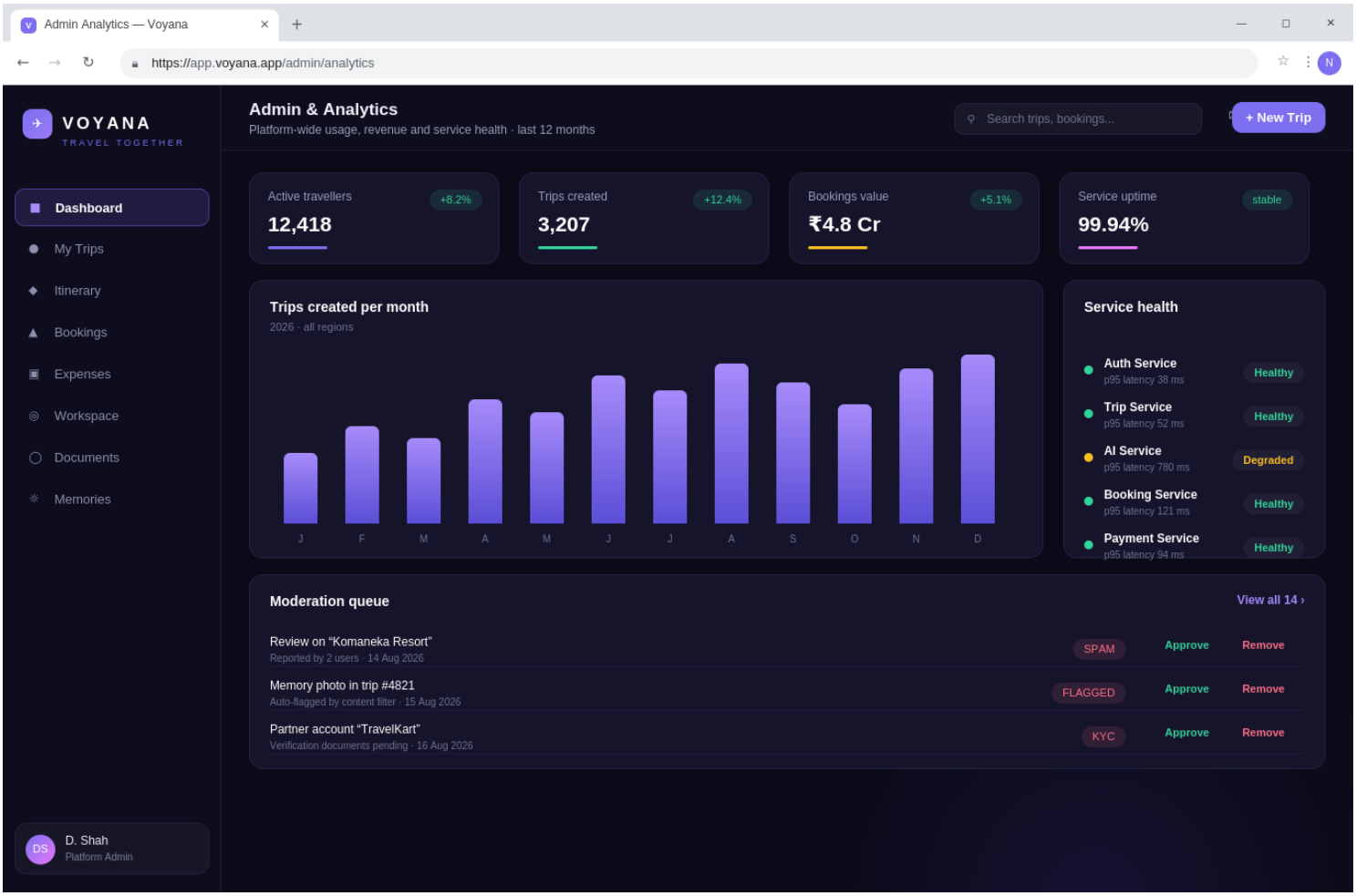


Figure 21. Administration and Analytics Console - final mockup.

Design elements	Services and endpoints behind the screen
KPI cards, monthly volume chart, per-service health list with p95 latency, moderation queue with reason chips.	GET /admin/health, GET /admin/moderation, POST /admin/moderation/{id}/decision. Every decision writes to audit_log.

5.9 Responsive Behaviour

Screen	360 px (mobile)	768 px (tablet)	1024 px and above
Landing	Globe collapses to a static hero image; search moves above the fold; cards stack.	Globe at 60 percent scale beside a single result column.	As mocked up.
Dashboard	Trip cards stack; metrics become a two-column grid; feed truncates to five items.	Trip cards in two columns; metrics in four.	As mocked up.
Itinerary	Assistant collapses to a floating launcher; day selector scrolls horizontally.	Assistant becomes a bottom sheet; activity list full width.	Two columns as mocked up.
Workspace	Chat, polls and tasks become three tabs.	Chat full width; polls and tasks in an accordion below.	Two columns as mocked up.
Expenses	Budget bar pinned to top; log as compact rows; settlements in a tab.	Log full width; settlements and chart below.	Two columns as mocked up.
Booking	Criteria collapse to a summary chip; summary becomes a sticky footer bar.	Results full width; summary as a sticky footer.	Two columns as mocked up.
Admin	Not supported below 768 px; a message directs the administrator to a larger display.	Metrics stack; moderation queue as cards.	As mocked up.